

1. Title Page

Lab Name: Lab 04: DNS Foundations with CML and Wireshark **Date:** YYYY-MM-DD **Name:** [Your Name] **Lab Partners:** [Partners' Names, if any] **Software/Hardware Used:** Cisco Modeling Labs (CML) - Personal / Free Edition

2. Background

The Domain Name System (DNS) is a critical Internet service that translates human-readable domain names (like `www.example.com`) into numerical IP addresses, allowing computers to locate each other on a network. Sockets provide the foundational programming interface that applications use to communicate over a network via TCP or UDP. Understanding how DNS resolves names and how sockets establish connections is essential for network application development. In this lab, you will explore DNS query and response mechanisms and utilize basic socket programming concepts within your CML environment to bridge the gap between network applications and underlying transport protocols.

3. Objective / Purpose

To observe DNS resolution traffic and understand the Domain Name System (DNS) by setting up a local DNS server using `dnsmasq`, querying it using `nslookup` (or `dig`), and capturing packets with Wireshark via the Cisco Modeling Labs (CML) UI.

4. Topology Diagram

```
graph TD
    %% Define Nodes
    subgraph Local_Subnet ["Local Isolated Subnet (192.168.10.0/24)"]
        H1["Client Node (H1)<br>OS: Alpine Linux<br>eth0: 192.168.10.10"]
        H2["DNS Server Node (H2)<br>OS: Ubuntu Server<br>eth0: 192.168.10.20<br>Services: dnsmasq"]
    end

    %% Link & Packet Capture
    H1 <--> |"eth0 <--> eth0<br>[Packet Capture Filter: udp port 53]"| H2
    end

    %% External Internet Connection
    subgraph External_Network ["External Network Access (Temporary/NAT)"]
        ExtConn["CML External Connector<br>(NAT Mode)"]
    end
```

```
end

H2 <--> |"eth1 (DHCP)<br>[Download dnsmasq via apt]"| ExtConn

%% Styling
classDef client fill:#d4f1f9,stroke:#007b8b,stroke-width:2px;
classDef server fill:#e2f0d9,stroke:#385723,stroke-width:2px;
classDef connector fill:#fff2cc,stroke:#d6b656,stroke-width:2px;

class H1 client;
class H2 server;
class ExtConn connector;
```

Details: - Client Node: Alpine Linux (H1) - DNS Server Node: Ubuntu Server (H2) - Subnet: 192.168.10.0/24

5. Equipment & Environment

- **Software:** Cisco Modeling Labs (CML), Terminal Emulator (e.g., PuTTY, SecureCRT, native terminal), Wireshark
 - **Hardware / Virtual Nodes:** 1x Alpine Linux Node (Client), 1x Ubuntu Server Node (DNS Server), 1x CML External Connector (NAT mode)
-

6. Configuration / Methodology

IP Addressing Table

Device	Node Type	Interface	IP Address	Subnet Mask
Client (H1)	Alpine Linux	eth0	192.168.10.10	255.255.255.0
Server (H2)	Ubuntu Server	eth0	192.168.10.20	255.255.255.0

Connecting the Server Node to the External World (Internet Access in CML)

Since your nodes reside in a private subnet, they do not have direct internet access to download and install dnsmasq. To enable internet access, you must temporarily connect your **Server Node (H2 - Ubuntu)** to CML's **External Connector** in **NAT** mode:

1. Drag an **External Connector** node from the CML palette onto the workspace. Ensure its **Connection Mode** is set to **NAT** in the properties pane.
2. Connect a link from the External Connector to interface eth1 on the Server Node (leaving eth0 connected to the Client H1 on your local 192.168.10.0/24 subnet).
3. Start the External Connector and open the Server Node's console.
4. Run the DHCP client or configure the IP parameters manually to enable internet connectivity:

Method A: Automatic Configuration via DHCP (Recommended)

DHCP automatically configures the IP address, default gateway, and DNS servers:

```
sudo dhclient eth1
```

Method B: Manual (Static) Configuration of Gateway and DNS

If assigning static IP parameters on the external bridge subnet (e.g., NAT subnet 192.168.255.0/24 with gateway 192.168.255.1):

1. **Verify your interface is active:**

```
ip addr show dev eth1
```

2. **Configure the Default Gateway:**

```
sudo ip route add default via 192.168.255.1 dev eth1
```

3. **Configure the DNS Server:**

```
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
```

5. Once connected, update the Ubuntu package lists:

```
sudo apt update
```

Setting up the DNS Server (dnsmasq on Ubuntu Server H2)

First, configure your Server Node to act as a local DNS Server for your environment.

1. Connect to the Server Node console.
2. Install dnsmasq on the Ubuntu Server:

```
bash sudo apt install -y dnsmasq
```
3. Edit the local DNS records. Open /etc/hosts and add the following lines at the bottom:

```
192.168.10.20 ns1.example.local
192.168.10.100 www.example.local
```
4. Restart the dnsmasq service to apply the changes:

```
bash sudo systemctl restart dnsmasq
```

Capturing Packets via CML UI

In this lab, you will generate traffic on your Client Node and capture it directly from the link in CML.

1. Open your CML Dashboard.
2. Right-click on the link connecting your Client Node to the network (or directly to the Server Node).
3. Select **Packet Capture**. Set the filter if necessary (e.g., `udp port 53`) and click **Start**.
4. Generate the DNS traffic from your Client Node as instructed in the next sections.
5. Once complete, stop the capture in the CML UI and click **Download PCAP**.

6. Open the downloaded .pcap file locally on your computer using Wireshark to answer the questions.

1. nslookup

Let's investigate DNS by examining the nslookup command from your Client Node. Point it to your newly created DNS server.

```
nslookup www.example.local 192.168.10.20
```

In its most basic operation, nslookup allows the host running nslookup to query a specified DNS server for a DNS record. For example, nslookup can retrieve a "Type=A" DNS record that maps a hostname to its IP address.

Note: The images below are examples from public queries, but you will perform similar actions against your local example.local domain.

```
[newworld.cs.umass.edu> nslookup www.nyu.edu
Server:          128.119.240.1
Address:         128.119.240.1#53

Non-authoritative answer:
www.nyu.edu      canonical name = WEB.GSLB.nyu.edu.
Name:   WEB.GSLB.nyu.edu
Address: 216.165.47.12
Name:   WEB.GSLB.nyu.edu
Address: 2607:f600:1002:6113::100
```

Figure 1: the basic nslookup command (example)

We can also use nslookup to query for a "TYPE=NS" record, which returns the hostname of an authoritative DNS server.

```
newworld.cs.umass.edu> nslookup -type=NS nyu.edu
Server:          128.119.240.1
Address:         128.119.240.1#53
[
Non-authoritative answer:
nyu.edu nameserver = ns2.nyu.org.
nyu.edu nameserver = ns4.nyu.edu.
nyu.edu nameserver = ns1.nyu.net.

Authoritative answers can be found from:
ns2.nyu.org      internet address = 128.122.0.76
ns1.nyu.net      internet address = 128.122.0.8
ns4.nyu.edu      internet address = 216.165.87.102
ns4.nyu.edu      has AAAA address 2607:f600:2001:6100::135
```

Figure 2: using nslookup to find authoritative name servers (example)

nslookup can also be used to perform a "reverse DNS lookup" by specifying an IP address.

```
[kurose@MacBook-Pro-6 ~ % nslookup 128.119.245.12
Server:          75.75.75.75
Address:         75.75.75.75#53

Non-authoritative answer:
12.245.119.128.in-addr.arpa      name = gaia.cs.umass.edu.

Authoritative answers can be found from:
```

Figure 3: using nslookup to perform a "reverse DNS lookup" (example)

Now it's time for you to test drive it yourself. Start your packet capture in CML, then run the following on your Client Node:

1. Run `nslookup` to obtain the IP address of `www.example.local` using your server `192.168.10.20`. What is the IP address returned?
2. What is the IP address of the DNS server that provided the answer to your command in question 1 above?
3. Use the `nslookup` command to query the `example.local` domain.
Note: dnsmasq might not return a full NS record by default without extra configuration, but observe the response. What information is returned?

2. The DNS cache on your Client Node

Most desktop operating systems keep a cache of recently retrieved DNS records. However, because your Client Node H1 is running **Alpine Linux**, it does not utilize a local DNS caching daemon (such as `systemd-resolved` or `nscd`) by default.

On Alpine Linux, every DNS query is sent directly to the configured DNS server without checking a local cache. Therefore, **no cache flushing commands are needed** on the Client Node for this lab.

3. Tracing DNS with Wireshark

Now let's look at the captured DNS messages.

- (Skip if using Alpine Linux Client) Ensure your DNS cache is cleared on the Client Node.
- Start packet capture in the CML UI on the link.
- Perform a lookup for `www.example.local` and another external lookup if your lab has internet access (e.g., `nslookup www.cisco.com 192.168.10.20`).
- Stop packet capture and download the PCAP. Open it in Wireshark.

Answer the following questions based on your capture:

5. Locate the first DNS query message for `www.example.local`. What is the packet number in the trace for the DNS query message? Is this query message sent over UDP or TCP?
6. Now locate the corresponding DNS response. What is the packet number in the trace for the DNS response message? Is this response message received via UDP or TCP?
7. What is the destination port for the DNS query message? What is the source port of the DNS response message?
8. To what IP address is the DNS query message sent?
9. Examine the DNS query message. How many "questions" does this DNS message contain? How many "answers" does it contain?
10. Examine the DNS response message to the initial query message. How many "questions" does this DNS message contain? How many "answers" does it contain?

The image shows a Wireshark packet capture titled 'dns-wireshark-trace-2.pcapng'. The packet list pane shows four DNS packets:

No.	Time	Source	Destination	Protocol	Length	Info
19	6.003804	10.0.0.44	75.75.75.75	DNS	76	Standard query 0x609b A www.cs.umass.edu
20	6.037987	75.75.75.75	10.0.0.44	DNS	92	Standard query response 0x609b A www.cs.umass.edu A 128.119.240.84
31	10.494907	10.0.0.44	75.75.75.75	DNS	80	Standard query 0x1462 A cc-api-data.adobe.io
32	10.512877	75.75.75.75	10.0.0.44	DNS	208	Standard query response 0x1462 A cc-api-data.adobe.io A 52.5.6.70 A 52.7.200.11

The packet details pane for packet 19 shows:

- Frame 19: 76 bytes on wire (608 bits), 76 bytes captured (608 bits) on interface en0, id 0
- Ethernet II, Src: Apple_98:d9:27 (78:4f:43:98:d9:27), Dst: Intel_80:00:00 (00:50:f1:80:00:00)
- Internet Protocol Version 4, Src: 10.0.0.44, Dst: 75.75.75.75
- User Datagram Protocol, Src Port: 57837, Dst Port: 53
- Domain Name System (query)
 - Transaction ID: 0x609b
 - Flags: 0x0100 Standard query
 - Questions: 1
 - Answer RRs: 0
 - Authority RRs: 0
 - Additional RRs: 0
 - Queries
 - [Response In: 20]

The packet bytes pane shows the raw data in hexadecimal and ASCII:

```

0000  00 50 f1 80 00 00 78 4f 43 98 d9 27 00 00 45 00  P....x0 C...E-
0010  00 3e c2 aa 00 00 40 11 17 43 0a 00 00 2c 4b 4b  >...@...C...KK
0020  4b 4b e1 ed 00 35 00 2a 06 77 68 9b 01 00 00 01  KK...S...w....
0030  00 00 00 00 00 00 03 77 77 77 02 63 73 05 75 6d  ....w ww cs um
0040  61 73 73 03 65 64 75 00 00 01 00 01             ass.edu ....

```

Figure 5: Wireshark trace for nslookup (example)

Examine a DNS query that your server had to forward or resolve recursively (if internet is available), or simply examine your local queries:

- What is the destination port for the DNS query message? What is the source port of the DNS response message?
- To what IP address is the DNS query message sent? Is this the IP address of your configured local DNS server (192.168.10.20)?
- Examine the DNS query message. What “Type” of DNS query is it? Does the query message contain any “answers”?
- Examine the DNS response message to the query message. How many “questions” does this DNS response message contain? How many “answers”?

7. Verification & Testing

- Screenshot of the CML UI showing the packet capture running.
- Screenshot of Wireshark showing DNS Standard Query and Response from your downloaded .pcap.
- Provide answers to the questions listed in the text above.

8. Troubleshooting

Issue Encountered: [Describe what went wrong, if anything]

Discovery Method: [How did you find the issue?] **Resolution:** [How did you fix it?]

(If no issues were encountered, explain potential pitfalls for this configuration).

9. Conclusion & Analysis

Verified that DNS uses UDP port 53 and maps hostnames to IP addresses. Successfully set up a local dnsmasq server, generated DNS traffic, captured packets via CML, and analyzed the downloaded .pcap using Wireshark.

10. What to Hand In

- The Lab YAML file with your name, date, name of the lab at the top of the file in comments.
- A PDF with annotated screenshots showing the lab running, as well as screenshots of your Wireshark analysis, and the answers to the questions.